Тестирование и контроль качества

Проект должен сопровождаться документированным процессом тестирования.

Цель — доказать, что отдельные части кода работают корректно изолированно, а приложение в целом стабильно при взаимодействии с пользователем.

1. **Тест-план**

В проекте должен быть текстовый план тестирования, содержащий:

- Объекты тестирования: Выбор ключевых функций (например: **calculate_score**, **validate_login**, **save_game**).
- Тестовые данные: Описание входных данных, которые будут использоваться (корректные, некорректные, граничные).
- Ожидаемые результаты: Что должно произойти на выходе (возврат значения, выброс исключения, создание файла).

2. Модульные тесты

Тесты проверяют логику работы приложения в изоляции от внешнего мира (экрана, клавиатуры, жесткого диска).

- Фреймворк: Использование **pytest**.
- Охват тестов (Coverage): Минимум 4–5 тестов на функцию/метод.
    - Обязательна проверка позитивного сценария (успех).
    - Обязательна проверка негативных сценариев (обработка ошибок, неверный ввод).
    - Обязательна проверка граничных условий (например: 0 денег, пустой список, максимальное значение).

3. Интеграционные тесты и работа с данными

Проверка того, что модули корректно обмениваются данными.

- Тест "Сохранение-Загрузка": Тест должен записать данные в файл/БД, затем прочитать их и убедиться, что данные не исказились.
- Изоляция файловой системы: Тесты не должны портить рабочие файлы пользователя.
    - *Требование:* Использовать временные файлы или тестовую базу данных, которая удаляется или сбрасывается после завершения теста. Рабочий файл **data.json** проекта не должен меняться в процессе прогонки тестов.

4. Чек-лист ручного тестирования

Для проверки интерфейса и сценариев использования составляется таблица. Студент должен пройти по пунктам и отметить статус (Пройдено/Не пройдено).

Обязательные проверки:

1. Стабильность (Robustness): Ввод букв в числовые поля, нажатие "Отмена" в диалогах ввода — приложение не должно вылетать с traceback.
2. Валидация UI: Попытка сохранить пустую форму или некорректный email блокируется с сообщением об ошибке.
3. Постоянство данных (Persistence):
   - Запустить приложение -> Создать запись -> Выйти.
   - Запустить приложение снова -> Запись на месте.
4. Игровые сценарии (для игр/викторин):
   - Проверка состояния "Game Over" (проигрыш) и "Win" (выигрыш).
   - Корректный перезапуск игры (сброс очков и поля) без закрытия приложения.

5. **Документирование и запуск**

- Тесты должны лежать в отдельном модуле (например, **tests.py** или в папке **tests/**).
- В коде тестов должны быть комментарии (docstrings), объясняющие, что именно проверяется.
- Должна быть команда для быстрого запуска всех тестов (через терминал или IDE).

ТЕСТ-ПЛАН ИГРЫ "ВЫЖИТЬ ОБЯЗАН"

1. ОБЩАЯ ИНФОРМАЦИЯ

Параметр	Значение
Название проекта	Выжить обязан
Тип приложения	Десктопная игра на Tkinter
Версия	1.0
Дата тестирования	15.03.2026
Тестировщик	Команда разработки
Всего тестов	45

2. ОБЪЕКТЫ ТЕСТИРОВАНИЯ

2.1. Модуль вспомогательных функций (Helper Functions)

Файл**:** Test_play.py

Тестируемая функция**:** split_long_text(text, max_length=20)

№	Название теста	Что проверяем	Тип теста
1	test_split_long_text_short	Разбиение короткого текста (≤20 символов)	Позитивный

2	test_split_long_text_long	Разбиение длинного текста (>20 символов)	Позитивный
3	test_split_long_text_empty	Разбиение пустого текста	Граничный
4	test_split_long_text_exact_length	Текст ровно 20 символов	Граничный
5	test_split_long_text_one_long_word	Одно очень длинное слово	Негативный
6	test_split_long_text_special_chars	Текст со спецсимволами	Позитивный

2.2. Модуль игры в пятнашки (WirePuzzle)

Файл: Test_play.py

Класс: WirePuzzle

№	Название теста	Что проверяем	Тип теста
7	test_puzzle_initialization	Создание окна и инициализация доски	Позитивный
8	test_move_valid_adjacent	Перемещение соседней с пустой плитки	Позитивный
9	test_move_invalid_not_adjacent	Перемещение несоседней плитки	Негативный
10	test_move_first_tile	Попытка переместить первую (закрепленную) плитку	Негативный
11	test_move_when_empty_at_edge	Пустая клетка на границе (в углу)	Граничный
12	test_multiple_moves	Несколько перемещений подряд	Интеграционный
13	test_get_image_with_value	Получение изображения для плитки с числом	Позитивный
14	test_get_image_with_none	Получение изображения для пустой клетки	Граничный

2.3. Модуль просмотра документов (DocumentViewer)

Файл: Test_play.py

Класс: DocumentViewer

№	Название теста	Что проверяем	Тип теста
15	test_document_viewer_initialization	Создание окна с тремя документами	Позитивный
16	test_choose_document_2_success	Выбор документа №2 с подтверждением	Позитивный
17	test_choose_document_2_cancel	Отмена выбора документа №2	Негативный
18	test_choose_document_1	Выбор документа №1 (неправильного)	Негативный
19	test_choose_document_3	Выбор документа №3 (неправильного)	Негативный
20	test_view_document_all	Просмотр всех трех документов	Интеграционный

2.4. Основной класс игры (Game) - базовые методы

Файл: Test_play.py

Класс: Game

№	Название теста	Что проверяем	Тип теста
21	test_game_initialization	Инициализация игры (переменные состояния)	Позитивный
22	test_clear_window	Очистка окна от виджетов (кроме save_panel)	Позитивный
23	test_start_new_game	Сброс состояния при начале новой игры	Позитивный

2.5. Метод check_key() - проверка документов

Файл: Test_play.py

Метод: check_key(num)

№	Название теста	Входные данные	Ожидаемый результат	Тип теста
24	test_check_key_correct	num=2	has_basement_key = True	Позитивный
25	test_check_key_wrong_1	num=1	Вызов death_ending(), ключ не получен	Негативный
26	test_check_key_wrong_3	num=3	Вызов death_ending(), ключ не получен	Негативный
27	test_check_key_invalid_string	num="2" (строка)	Вызов death_ending()	Граничный
28	test_check_key_after_already_found	num=2, ключ уже есть	Состояние не меняется	Граничный

2.6. Метод save_game() - сохранение игры

Файл: Test_play.py

Метод: save_game()

№	Название теста	Что проверяем	Тип теста
29	test_save_game_success	Успешное сохранение с базовыми данными	Позитивный
30	test_save_game_permission_error	Ошибка прав доступа при сохранении	Негативный
31	test_save_game_disk_full	Ошибка "диск полон" при сохранении	Негативный
32	test_save_game_with_all_data	Сохранение со всеми возможными данными	Позитивный
33	test_save_game_empty_state	Сохранение пустого состояния (по умолчанию)	Граничный

2.7. Метод load_game() - загрузка игры

Файл: Test_play.py

Метод: load_game()

№	Название теста	Что проверяем	Тип теста
34	test_load_game_success	Успешная загрузка из корректного файла	Позитивный
35	test_load_game_file_not_found	Файл сохранения не найден	Негативный
36	test_load_game_corrupted_json	Поврежденный JSON в файле	Негативный
37	test_load_game_missing_keys	Отсутствуют некоторые ключи в JSON	Граничный
38	test_load_game_wrong_types	Неправильные типы данных в JSON	Негативный

2.8. Локации - навигация

Файл: Test_play.py

Метод: get_location_name()

№	Название теста	Что проверяем	Кол-во проверок	Тип теста
39	test_get_location_name_all	Названия всех 14 локаций	14	Позитивный

2.9. Диалоговая система

Файл: Test_play.py

Методы: next_dialogue_stage(), start_riddles()

№	Название теста	Что проверяем	Тип теста
40	test_dialogue_progression	Переход по этапам диалога (0→1→2→3)	Позитивный
41	test_dialogue_out_of_range	Невалидный этап диалога (999)	Граничный

2.10. Граничные случаи

Файл: Test_play.py

Различные методы

№	Название теста	Что проверяем	Тип теста
42	test_empty_password_parts	Пустой список частей пароля	Граничный
43	test_code_words_empty	Пустой словарь кодовых слов	Граничный
44	test_show_help	Показ окна справки	Позитивный

2.11. Интеграционный тест

Файл: Test_play.py

Полный игровой цикл

№	Название теста	Что проверяем	Кол-во шагов	Тип теста
45	test_full_game_flow	Полное прохождение игры от начала до хорошей концовки	13 шагов	Интеграционный

2.12. Методы навигации (Navigation)

Файл: `ugraaa.py`

Методы: `go_to_basement()`, `return_from_museum()`, `return_from_basement()`

№	Название теста	Что проверяем	Тип теста
46	test_go_to_basement	Переход в подвал с сохранением предыдущей локации	Позитивный
47	test_return_from_museum_to_second_floor_2	Возврат из музея на второй этаж	Позитивный
48	test_return_from_museum_to_second_floor_1	Возврат из музея в начало коридора	Позитивный
49	test_return_from_basement_to_second_floor_2	Возврат из подвала на второй этаж	Позитивный

3. ТЕСТОВЫЕ ДАННЫЕ

3.1. Корректные данные

Категория	Данные	Где используется
Текст для разбиения	"Короткий текст"	split_long_text()
Текст для разбиения	Длинный текст (50+ символов)	split_long_text()
Текст точной длины	"12345678901234567890" (20 символов)	split_long_text()
Текст со спецсимволами	"Текст с !@#$% и переносами\nстроки"	split_long_text()
Номер документа	2	check_key(), choose_document()
Данные сохранения	solved_puzzle=True, has_museum_key=True	save_game(), load_game()
Локации	"first_floor", "room_22", "basement"	get_location_name()

3.2. Некорректные данные

Категория	Данные	Ожидаемая реакция	Где используется
Очень длинное слово	"Оченьдлинноесловокотороеневлезает"	Разбиение на части	split_long_text()
Номер документа	1	Смерть игрока	check_key()

Номер документа	3	Смерть игрока	check_key()
Строка вместо числа	"2"	Смерть игрока	check_key()
Ошибка прав доступа	PermissionError	Сообщение об ошибке	save_game()
Ошибка диска	OSError	Сообщение об ошибке	save_game()
Поврежденный JSON	"{-invalid-json}"	Сообщение об ошибке	load_game()
Неправильные типы	solved_puzzle="True" (строка)	Обработка без падения	load_game()

3.3. Граничные значения

Параметр	Значение	Где используется
Максимальная длина строки	20 символов	split_long_text()
Пустой текст	""	split_long_text()
Индекс плитки	0 (первая)	move()
Индекс плитки	7 (последняя перед пустой)	move()
Пустая клетка	None в board	get_image()
Пустой список	password_parts = []	room_22()
Пустой словарь	code_words = {}	show_password_window()
Этап диалога	999	next_dialogue_stage()
Отсутствие файла	FileNotFoundError	load_game()
Отсутствие ключей в JSON	Неполный словарь	load_game()

4. ОЖИДАЕМЫЕ РЕЗУЛЬТАТЫ

4.1. Возврат значений

Метод/Функция	Входные данные	Ожидаемое возвращаемое значение
split_long_text()	Короткий текст	Текст без переносов
split_long_text()	Длинный текст	Текст с переносами \n
split_long_text()	Пустой текст	""
get_image()	Индекс с числом	PhotoImage объект
get_image()	Индекс с None	empty_image
get_location_name()	"first_floor"	"Первый этаж"
get_location_name()	"room_22"	"Компьютерный класс"

4.2. Изменение состояния

Действие	Что меняется	Новое значение
check_key(2)	has_basement_key	True
puzzle_complete()	solved_puzzle	True
find_museum_key()	has_museum_key	True
start_new_game()	Все переменные	Сброс к False/[]/{}
next_dialogue_stage()	dialogue_stage	stage + 1
move(7) при empty_idx=8	board[7] и board[8]	Меняются местами
move(7) при empty_idx=8	empty_idx	7

4.3. Вызов функций

Действие	Какая функция вызывается	Сколько раз
check_key(1)	death_ending()	1
check_key(3)	death_ending()	1
save_game() успех	messagebox.showinfo()	1
save_game() ошибка	messagebox.showerror()	1
load_game() успех	перейти на сохраненную локацию	1
load_game() ошибка	messagebox.showerror()	1
choose_document(2) с подтверждением	on_choice(2)	1
choose_document(2) с отменой	on_choice()	0

4.4. Создание/изменение файлов

Действие	Файл	Действие	Проверка
save_game()	save.json	Создание/перезапись	mock_file.assert_called_once()
save_game()	save.json	Запись JSON	mock_json_dump.assert_called_once()
load_game()	save.json	Чтение	mock_open используется

4.5. Исключения

Ситуация	Исключение	Обработка
Нет файла сохранения	FileNotFoundError	Показ сообщения, игра не падает
Поврежденный JSON	json.JSONDecodeError	Показ сообщения, игра не падает
Нет прав на запись	PermissionError	Показ сообщения, игра не падает
Диск полон	OSError	Показ сообщения, игра не падает
Нет изображения	FileNotFoundError	Создание заглушки, игра не падает

5. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

5.1. Общая статистика

Показатель	Значение
Всего тестов	45
Успешно пройдено	49 (100%)
Провалено	0
Время выполнения	10.32 секунды

5.2. Распределение по категориям

Категория	Кол-во тестов	Статус	Покрытие требований
Вспомогательные функции	6	✓	Позитивные, негативные, граничные
Пятнашки (WirePuzzle)	8	✓	Позитивные, негативные, граничные, интеграционный
Просмотр документов	6	✓	Позитивные, негативные, интеграционный
Основная игра (базовые)	3	✓	Позитивные
Метод check_key()	5	✓	Позитивные, негативные, граничные
Метод save_game()	5	✓	Позитивные, негативные, граничные
Метод load_game()	5	✓	Позитивные, негативные, граничные
Методы навигации	4	✓	Позитивные
Локации	1 (14 проверок)	✓	Позитивные
Диалоги	2	✓	Позитивные, граничные
Граничные случаи	3	✓	Граничные, позитивные
Интеграционный тест	1 (13 шагов)	✓	Полный игровой цикл

5.3. Соответствие требованиям

Требование	Выполнение	Комментарий
Модульные тесты	✓	45 тестов для всех ключевых функций
Минимум 4-5 тестов на функцию	✓	split_long_text (6), move() (5), check_key() (5), save_game() (5),
Позитивные сценарии	✓	Проверены во всех модулях
Негативные сценарии	✓	Неправильные документы, ошибки сохранения, поврежден
Граничные условия	✓	Пустые значения, крайние индексы, точная длина, невалид
Интеграционные тесты	✓	Сохранение-загрузка, полный игровой цикл, несколько ходов
Изоляция файловой системы	✓	Использование mock_open и временных файлов
Документирование	✓	Docstrings для каждого теста

6. ВЫВОД

Тестирование завершено успешно!

Все 49 тестов пройдены, что подтверждает:

1. Корректную работу всех ключевых функций игры
2. Устойчивость к некорректным данным
3. Правильную обработку граничных случаев

4. Надежную систему сохранения/загрузки
5. Стабильность при полном прохождении игры